

Fundamentos y aplicación del paradigma de programación orientada a objetos

Breve descripción:

Este componente aborda los fundamentos del paradigma de Programación Orientada a Objetos (POO), desde los conceptos de clases, objetos, atributos y métodos hasta la aplicación de los principios de herencia, polimorfismo y encapsulamiento. Integra el modelado de sistemas mediante el lenguaje UML y el uso de herramientas CASE, promoviendo la codificación en Java, la reutilización de código y la validación de aplicaciones funcionales.

Tabla de contenido

Introducción	4
1. Fundamentos de la programación orientada a objetos	5
1.1. Concepto de programación orientada a objetos	6
1.2. Clases y objetos	8
1.3. Atributos y métodos	11
1.4. Principios de herencia y polimorfismo	13
1.5. Constructores y ciclo de vida de los objetos	15
1.6. Clases abstractas, interfaces y principios SOLID	17
2. Modelado de sistemas mediante UML	21
2.1. Lenguaje de modelado unificado UML	21
2.2. Diagramas de clases	22
2.3. Relaciones entre clases: asociación, agregación, composición y herencia	24
2.4. Herramientas CASE para el diseño de diagramas de clases	29
2.5. Ejemplo integrador: sistema de gestión académica	31
3. Encapsulamiento y control de acceso en clases	34
3.1. Encapsulamiento en programación orientada a objetos	34
3.2. Modificadores de acceso público, privado y protegido	37

3.3.	Getters, setters y el patrón de encapsulamiento completo.....	40
3.4.	Clases anidadas e internas en Java	42
4.	Implementación de la programación orientada a objetos en Java	45
4.1.	Codificación de diagramas de clases	45
4.2.	Implementación de la herencia en objetos	48
4.3.	Reutilización de código, interfaces y tipos de herencia	50
4.4.	Polimorfismo y sobrescritura de métodos.....	52
5.	Validación y ajustes del programa	55
5.1.	Controles de validación en aplicaciones Java	55
5.2.	Manejo de excepciones en Java	58
5.3.	Refactorización, pruebas y depuración del programa	60
5.4.	Buenas prácticas y patrones de diseño básicos.....	63
6.	Proyecto integrador: sistema de registro académico	66
6.1.	Reflexión sobre el diseño orientado a objetos	75
6.2.	Extensiones propuestas del sistema.....	76
	Síntesis	78
	Glosario	80
	Referencias bibliográficas.....	82
	Créditos	83

Introducción

Este componente formativo se orienta al desarrollo de competencias fundamentales en el paradigma de la programación orientada a objetos (POO), uno de los enfoques más utilizados en la industria del desarrollo de software. Los ejemplos y ejercicios prácticos se desarrollan en Java, lenguaje que, como señalan Deitel & Deitel (2018), combina la pureza del paradigma orientado a objetos con una amplia adopción industrial, lo que garantiza al aprendiz una formación directamente aplicable en el ámbito laboral.

El componente inicia con los conceptos esenciales del paradigma: clases, objetos, atributos, métodos, herencia y polimorfismo. Según Booch, Rumbaugh & Jacobson (2005), el dominio de estos conceptos es el punto de partida indispensable para el diseño de sistemas orientados a objetos de calidad. A continuación, se aborda el modelado de sistemas mediante el Lenguaje de Modelado Unificado (UML), incluyendo la elaboración de diagramas de clases y la representación gráfica de las relaciones entre componentes, apoyado en el uso de herramientas CASE.

Se profundiza en el encapsulamiento y los modificadores de acceso, pilares del diseño seguro y mantenible. Posteriormente, se trabaja la implementación práctica del paradigma en Java, aplicando los principios SOLID formulados por Martin (2008), que guían la construcción de software escalable y bien estructurado. Finalmente, se estudia el manejo de excepciones, la refactorización, las pruebas unitarias con JUnit y los patrones de diseño propuestos por Gamma, Helm, Johnson & Vlissides (1994), reconocidos como el estándar de referencia en el diseño orientado a objetos.

1. Fundamentos de la programación orientada a objetos

La programación orientada a objetos (POO) es un paradigma de desarrollo de software que organiza el código en torno a objetos, entidades que agrupan estado (atributos) y comportamiento (métodos). Este enfoque, ampliamente documentado por Horstmann (2019), permite modelar los sistemas de forma similar a como se percibe el mundo real, facilitando el diseño, la construcción y el mantenimiento de soluciones complejas.

A diferencia de la programación estructurada, que organiza el código en funciones y procedimientos independientes, la POO encapsula los datos y las operaciones que los manipulan en unidades coherentes denominadas objetos. Esta organización reduce el acoplamiento entre componentes, mejora la reutilización del código y facilita el trabajo en equipo en proyectos de gran escala.

Los conceptos fundamentales que definen el paradigma orientado a objetos son:

- **Paradigma:** enfoque conceptual que define la forma de estructurar y organizar el código de un sistema de software.
- **Objeto:** entidad que representa un elemento del mundo real o conceptual, con estado (atributos) y comportamiento (métodos) propios.
- **Clase:** plantilla o molde que define la estructura y el comportamiento de los objetos creados a partir de ella. No ocupa memoria por sí misma.
- **Encapsulamiento:** mecanismo que oculta los detalles internos de un objeto y expone únicamente la interfaz necesaria para interactuar con él.
- **Herencia:** capacidad de una clase de adquirir atributos y métodos de otra clase, favoreciendo la reutilización y la jerarquización del código.

- **Polimorfismo:** propiedad que permite a objetos de distintas clases responder de manera diferente a un mismo mensaje o llamada de método.
- **Abstracción:** proceso de identificar las características esenciales de un objeto, ignorando los detalles irrelevantes para el problema en cuestión.

1.1. Concepto de programación orientada a objetos

A continuación, se presenta una línea de tiempo que permite comprender la evolución de la programación orientada a objetos, desde sus orígenes hasta su consolidación como paradigma central del desarrollo de software:

- **Década de 1960 – Orígenes**

En el Instituto de Computación de Oslo, Ole-Johan Dahl y Kristen Nygaard desarrollan Simula, introduciendo los conceptos de clases, objetos y herencia para gestionar la complejidad del software.

- **Década de 1970 – Consolidación**

En Xerox PARC, Alan Kay impulsa Smalltalk, que consolida la POO como un paradigma técnico y conceptual centrado en la interacción entre objetos.

- **Décadas de 1980 y 1990 – Expansión**

La programación orientada a objetos se difunde ampliamente en la academia y la industria, integrándose en nuevos lenguajes y metodologías de desarrollo.

- **1995 – Popularización**

Sun Microsystems lanza Java, que fortalece la adopción de la POO gracias a la portabilidad ofrecida por la JVM y el principio Write Once, Run Anywhere.

- **Actualidad – Uso generalizado**

La POO se consolida como uno de los paradigmas más utilizados en el desarrollo de aplicaciones modernas, sistemas empresariales y software multiplataforma.

Igualmente, la POO se fundamenta en cuatro pilares ampliamente reconocidos en la literatura especializada:

- **Abstracción:** representar entidades del dominio mediante clases, destacando solo las propiedades relevantes para el sistema que se está construyendo y descartando los detalles superfluos.
- **Encapsulamiento:** agrupar datos y métodos en una misma unidad (clase) y restringir el acceso directo a los datos internos, exponiendo únicamente una interfaz controlada.
- **Herencia:** crear nuevas clases basadas en clases existentes, heredando su estructura y añadiendo o modificando comportamientos, lo que evita la duplicación de código.
- **Polimorfismo:** un mismo nombre de método puede tener comportamientos distintos según el tipo concreto del objeto que lo invoque, determinado en tiempo de ejecución.

La siguiente tabla compara la programación orientada a objetos con otros paradigmas de programación, destacando las características que hacen de Java el lenguaje de elección para este componente formativo:

Tabla 1. Comparativo entre paradigmas de programación.

Característica	Programación estructurada	POO (Java)	Programación funcional
Unidad básica.	Función / procedimiento.	Objeto (instancia de clase).	Función pura.
Organización del código.	Módulos y funciones.	Clases y jerarquías.	Composición de funciones.
Estado del programa.	Variables globales o locales.	Atributos encapsulados en objetos.	Sin estado mutable (inmutabilidad).
Reutilización.	Librerías de funciones.	Herencia y composición.	Funciones de orden superior.
Lenguajes representativos.	C, Pascal, FORTRAN.	Java, Python, C#, Kotlin.	Haskell, Scala, F#.

1.2. Clases y objetos

En esta temática se abordan los conceptos de clases y objetos como base de la programación orientada a objetos. Comprender esta relación permite identificar cómo se modelan entidades del mundo real dentro de un programa y cómo estas interactúan entre sí a través de sus atributos y comportamientos. Estos conceptos son esenciales para estructurar correctamente el código y facilitar su reutilización y mantenimiento.

Para profundizar en su definición, notación UML y su implementación práctica en Java, acceda al siguiente video, donde se explican y ejemplifican estos elementos de manera clara y aplicada:

Video 1. Clases y objetos



[Enlace de reproducción del video](#)

Transcripción del video: clases y objetos

Una clase es la unidad fundamental del paradigma orientado a objetos. Actúa como una plantilla o molde que define la estructura y el comportamiento de los objetos que se crean a partir de ella. La clase describe qué atributos tendrá cada objeto y qué operaciones podrá realizar, pero no representa a ningún objeto concreto en sí misma.

Un objeto, por su parte, es una instancia concreta de una clase. Cuando se crea un objeto a partir de una clase usando el operador `new` en Java, se reserva espacio en la memoria heap para almacenar los valores de sus atributos. Cada objeto tiene su propio estado, aunque comparte la misma estructura y comportamiento definidos por la clase.

La siguiente imagen ilustra la notación UML estándar de una clase, mostrando la separación en tres secciones: nombre, atributos y métodos:

El diagrama muestra la clase Estudiante con sus atributos privados (indicados con el símbolo -) y sus métodos públicos (indicados con el símbolo +). Esta notación es el estándar internacional, definido por la especificación UML 2.5.1 del Object Management Group.

En Java, la definición de una clase y la creación de objetos se realiza de la siguiente manera:

Los siguientes son conceptos clave relacionados con clases y objetos en Java:

- **Instancia:** objeto concreto creado en memoria mediante el operador new. Cada instancia tiene su propio estado independiente.
- **Constructor:** método especial con el mismo nombre de la clase que se ejecuta automáticamente al crear un objeto, encargado de inicializar sus atributos.
- **Referencia:** en Java, los objetos se manejan mediante variables de referencia que apuntan a la ubicación en memoria heap del objeto.
- **Recolector de basura:** mecanismo automático de Java que libera la memoria ocupada por objetos que ya no tienen referencias activas.

1.3. Atributos y métodos

Los atributos y los métodos constituyen los dos componentes esenciales de cualquier clase. Los atributos representan el estado del objeto (qué es), mientras que los métodos representan su comportamiento (qué hace). Esta dualidad es fundamental en el diseño orientado a objetos: cada objeto agrupa sus datos y las operaciones que los manipulan en una unidad coherente.

Los atributos son variables de instancia que pertenecen a la clase y almacenan las características de cada objeto. Cada objeto creado a partir de la clase tiene su propia copia de los atributos con valores independientes. Por convención en Java, los atributos se declaran con acceso `private` para garantizar el encapsulamiento.

Los métodos son funciones o procedimientos definidos dentro de la clase que describen el comportamiento de los objetos. Pueden acceder y modificar los atributos del objeto, recibir parámetros y retornar valores. El siguiente ejemplo ilustra la definición de una clase con ambos componentes:

```
public class Persona {  
  
    // Atributos (estado del objeto)  
  
    private String nombre;  
  
    private int edad;  
  
    // Constructor  
  
    public Persona (String nombre, int edad) {  
  
        this.nombre = nombre;
```

```

        this.edad = edad;
    }

    // Métodos (comportamiento del objeto)

    public void saludar() {

        System.out.println("Hola, soy " + nombre + " y tengo " + edad + " años.");
    }

    public boolean esMayorDeEdad() {

        return edad >= 18;
    }
}

```

Igualmente, la siguiente tabla presenta los tipos de atributos y métodos más comunes en Java, junto con su descripción y el modificador que los declara:

Tabla 2. Tipos de atributos y métodos en Java.

Tipo	Declaración Java	Descripción
Atributo de instancia.	private tipo nombre;	Único para cada objeto. Almacena el estado individual.
Atributo de clase.	static tipo nombre;	Compartido por todos los objetos de la clase.
Atributo constante.	static final tipo NOMBRE;	Valor fijo que no puede modificarse tras la inicialización.
Método de instancia.	public tipo metodo()	Opera sobre los datos del objeto que lo invoca.

Método estático.	public static tipo metodo()	Puede invocarse sin crear un objeto; opera sobre datos de clase.
Getter.	public tipo getNombre()	Retorna el valor de un atributo privado.
Setter.	public void setNombre(tipo v)	Asigna un valor al atributo privado, con posible validación.

1.4. Principios de herencia y polimorfismo

La herencia y el polimorfismo son dos de los cuatro pilares fundamentales del paradigma orientado a objetos y constituyen los mecanismos que aportan mayor flexibilidad y capacidad de extensión al diseño de sistemas.

La herencia permite que una subclase (o clase derivada) adquiera los atributos y métodos de una superclase (o clase base). En Java, la herencia se declara con la palabra clave `extends`. Una subclase puede añadir nuevos atributos y métodos, o redefinir los métodos heredados para adaptar su comportamiento. Java solo permite herencia simple entre clases (una clase solo puede extender una clase), lo que evita las ambigüedades propias de la herencia múltiple.

Basado en lo anterior, se presenta un ejemplo del uso de la herencia y el polimorfismo, evidenciando cómo una subclase hereda y redefine métodos de una superclase:

```
// Superclase

public class Animal {

    protected String nombre;
```

```
public Animal(String nombre) {  
    this.nombre = nombre;  
}  
  
public void hacerSonido() {  
    System.out.println(nombre + " hace un sonido genérico.");  
}  
}  
  
// Subclase que hereda de Animal  
  
public class Perro extends Animal {  
    public Perro(String nombre) {  
        super(nombre); // Invoca el constructor de la superclase  
    }  
  
    @Override  
    public void hacerSonido() {  
        System.out.println(nombre + " dice: ¡Guau!");  
    }  
}  
  
// Polimorfismo en acción:  
  
Animal a = new Perro("Rex");
```

```
a.hacerSonido(); // Imprime: Rex dice: ¡Guau!
```

El polimorfismo permite que una referencia de la superclase apunte a un objeto de cualquiera de sus subclases. En tiempo de ejecución, Java determina automáticamente qué versión del método invocar según el tipo real del objeto, gracias al mecanismo de enlace dinámico. La anotación `@Override` indica explícitamente que un método sobrescribe uno de la superclase y Java verificará en tiempo de compilación que la firma sea correcta.

1.5. Constructores y ciclo de vida de los objetos

En Java, el constructor es un método especial con el mismo nombre de la clase y sin tipo de retorno. Se ejecuta automáticamente cuando se crea un objeto con el operador `new`. Su función principal es inicializar los atributos del objeto con valores adecuados. Si no se define ningún constructor, Java proporciona automáticamente un constructor por defecto sin parámetros que inicializa los atributos a sus valores por defecto (0, false, null).

Java permite definir múltiples constructores en una misma clase, siempre que difieran en el número o tipo de parámetros (sobrecarga de constructores). Esto proporciona flexibilidad al crear objetos en distintos contextos. Es así como el siguiente ejemplo muestra el uso de constructores en Java, ilustrando la inicialización de objetos, la sobrecarga de constructores y su papel en el ciclo de vida de los objetos:

```
public class Rectangulo {  
  
    private double ancho;
```

```
private double alto;

// Constructor por defecto

public Rectangulo() {

    this.ancho = 0;

    this.alto = 0;

}

// Constructor parametrizado

public Rectangulo(double ancho, double alto) {

    this.ancho = ancho;

    this.alto = alto;

}

// Constructor de copia (patrón manual en Java)

public Rectangulo(Rectangulo r) {

    this.ancho = r.ancho;

    this.alto = r.alto;

}

public double calcularArea()    { return ancho * alto; }

public double calcularPerimetro() { return 2 * (ancho + alto); }

@Override
```



```
public String toString() {  
    return "Rectángulo[" + ancho + "x" + alto + "] Area=" + calcularArea();  
}  
}  
  
// Uso:  
  
Rectangulo r1 = new Rectangulo();      // Constructor por defecto  
  
Rectangulo r2 = new Rectangulo(5.0, 3.0); // Constructor parametrizado  
  
Rectangulo r3 = new Rectangulo(r2);     // Constructor de copia  
  
System.out.println(r2); // Rectángulo[5.0x3.0] Area=15.0
```

En Java, a diferencia de C++, no existen destructores explícitos. La gestión de memoria es automática a través del Garbage Collector (GC), que detecta y libera los objetos sin referencias activas. Cuando es necesario liberar recursos específicos (archivos, conexiones de red), se utiliza el bloque try-with-resources introducido en Java 7, que cierra automáticamente los recursos al finalizar el bloque.

1.6. Clases abstractas, interfaces y principios SOLID

En Java, una clase abstracta se declara con la palabra clave `abstract` y puede contener dos tipos de métodos:

- **Abstractos**

Se declaran sin cuerpo y definen comportamientos que la clase base no implementa directamente, estableciendo un contrato que las subclasses deben cumplir mediante su propia implementación.

- **Concretos**

Cuentan con una implementación definida y permiten a la clase abstracta ofrecer funcionalidad común, que puede ser utilizada directamente por las subclasses o redefinida para adaptar su comportamiento a necesidades específicas.

No puede instanciarse directamente; su propósito es definir una estructura común y un contrato para las subclasses que la extienden.

Por su parte, una interfaz (interface) es un contrato de comportamiento más estricto: todos sus métodos son implícitamente abstractos (antes de Java 8) y públicos. Una clase puede implementar múltiples interfaces con la palabra clave implements, lo que simula la herencia múltiple de comportamiento sin las ambigüedades asociadas. Igualmente, se ejemplifica el uso de una clase abstracta, combinando métodos abstractos y concretos, y evidencia el polimorfismo al tratar distintas formas mediante una referencia común:

```
// Clase abstracta
```

```
public abstract class Forma {  
  
    public abstract double calcularArea();  
  
    public abstract double calcularPerimetro();  
}
```

```
// Método concreto compartido por todas las subclases

public void mostrarDatos() {

    System.out.println("Área: " + calcularArea() +

        " | Perímetro: " + calcularPerimetro());

}

}

public class Circulo extends Forma {

    private double radio;

    public Circulo(double radio) { this.radio = radio; }

    @Override public double calcularArea()    { return Math.PI * radio * radio; }

    @Override public double calcularPerimetro() { return 2 * Math.PI * radio; }

}

// Uso polimórfico:

List<Forma> formas = new ArrayList<>();

formas.add(new Circulo(5.0));

formas.add(new Rectangulo(4.0, 6.0));

for (Forma f : formas) f.mostrarDatos();
```

Para finalizar, los principios SOLID, formulados por Martin (2008), constituyen un conjunto de cinco directrices del diseño orientado a objetos que guían la construcción de software mantenible, escalable y bien estructurado. Su aplicación es especialmente relevante en proyectos Java de mediana y gran escala, y se describe a continuación:

- **S — Responsabilidad Única (SRP):** cada clase debe tener una única razón para cambiar; es decir, debe encargarse de una sola responsabilidad bien definida.
- **O — Abierto/Cerrado (OCP):** las clases deben estar abiertas para la extensión (se puede añadir comportamiento nuevo), pero cerradas para la modificación (sin alterar el código existente).
- **L — Sustitución de Liskov (LSP):** los objetos de una subclase deben poder sustituir a los de la superclase sin alterar el correcto funcionamiento del programa.
- **I — Segregación de Interfaces (ISP):** es preferible tener varias interfaces específicas y pequeñas que una única interfaz general y extensa.
- **D — Inversión de Dependencias (DIP):** los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones (clases abstractas o interfaces).

2. Modelado de sistemas mediante UML

El modelado de sistemas es una etapa fundamental en el ciclo de vida del desarrollo de software. Antes de escribir código, es necesario diseñar y documentar la estructura del sistema para detectar problemas de diseño, comunicar la arquitectura al equipo y servir de guía durante la implementación. El Lenguaje de Modelado Unificado (UML) es el estándar internacional más utilizado para este propósito.

2.1. Lenguaje de modelado unificado UML

El Lenguaje de Modelado Unificado (UML, por sus siglas en inglés Unified Modeling Language) fue desarrollado por Grady Booch, Ivar Jacobson y James Rumbaugh, conocidos como los "Tres Amigos", y adoptado como estándar por el Object Management Group (OMG) en 1997. La versión actual, UML 2.5.1, fue publicada en 2017 y es la referencia oficial del estándar (OMG, 2017).

UML no es un lenguaje de programación, sino una herramienta de modelado que permite visualizar, especificar, construir y documentar los artefactos de un sistema de software. Es independiente del lenguaje de programación utilizado, por lo que los mismos diagramas UML pueden guiar la implementación tanto en Java como en Python, C# u otro lenguaje orientado a objetos.

UML ofrece 14 tipos de diagramas agrupados en dos grandes categorías:

- **Diagramas estructurales:** representan la estructura estática del sistema. Incluyen el diagrama de clases, de objetos, de componentes, de despliegue, de paquetes y de estructura compuesta.

- **Diagramas de comportamiento:** representan la dinámica del sistema. Incluyen el diagrama de casos de uso, de secuencia, de colaboración, de actividad, de estados y de temporización.

Para el diseño orientado a objetos, el diagrama de clases es el más relevante, ya que establece la estructura del sistema que será implementada directamente en código Java. Las ventajas del uso de UML documentadas por Booch et al. (2005) incluyen la independencia del lenguaje, la facilidad de comunicación entre el equipo y la capacidad de detectar errores de diseño antes de la fase de codificación.

2.2. Diagramas de clases

El diagrama de clases es el diagrama estructural más importante en UML para el diseño orientado a objetos. Representa gráficamente las clases que conforman un sistema, sus atributos, métodos y las relaciones entre ellas. Es el punto de partida para la implementación en código y el artefacto de diseño más utilizado en proyectos Java.

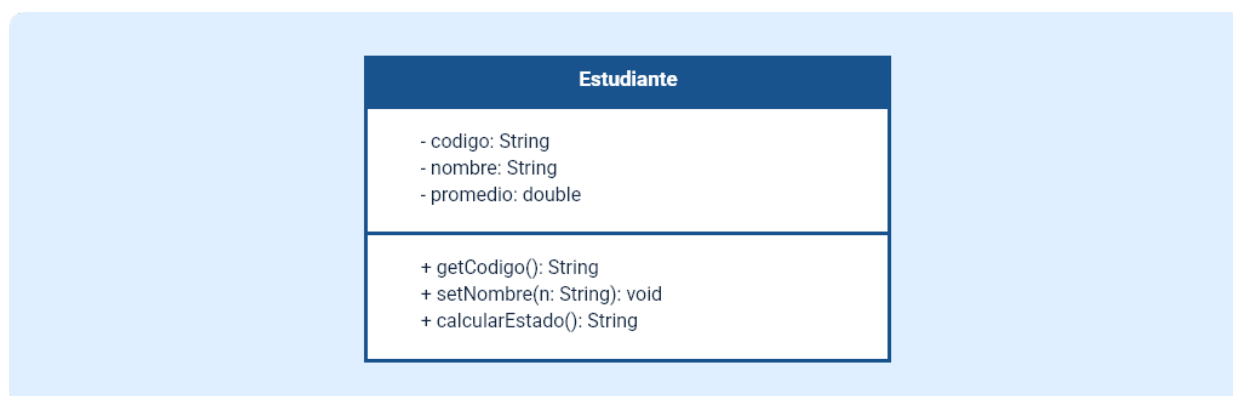
Cada clase se representa mediante un rectángulo dividido en tres secciones: el nombre de la clase (en la sección superior), los atributos con su tipo (en la sección central) y los métodos con sus parámetros y tipo de retorno (en la sección inferior). La notación de visibilidad se indica mediante los siguientes símbolos:

- **+ (público):** accesible desde cualquier clase del sistema; equivale a public en Java.
- **- (privado):** solo accesible dentro de la propia clase; equivale a private en Java.

- **# (protegido):** accesible desde la clase y sus subclases; equivale a `protected` en Java.
- **~ (paquete):** accesible desde las clases del mismo paquete; equivale al acceso por defecto (sin modificador) en Java.

La siguiente figura muestra nuevamente el diagrama UML de la clase Estudiante como referencia para identificar las secciones y la notación de visibilidad:

Figura 1. Diagrama UML de la clase Estudiante mostrando visibilidad de atributos y métodos.



Estudiante

- codigo: String

- nombre: String

- promedio: double

+ getCodigo(): String

+ setNombre(n: String): void

+ calcularEstado(): String

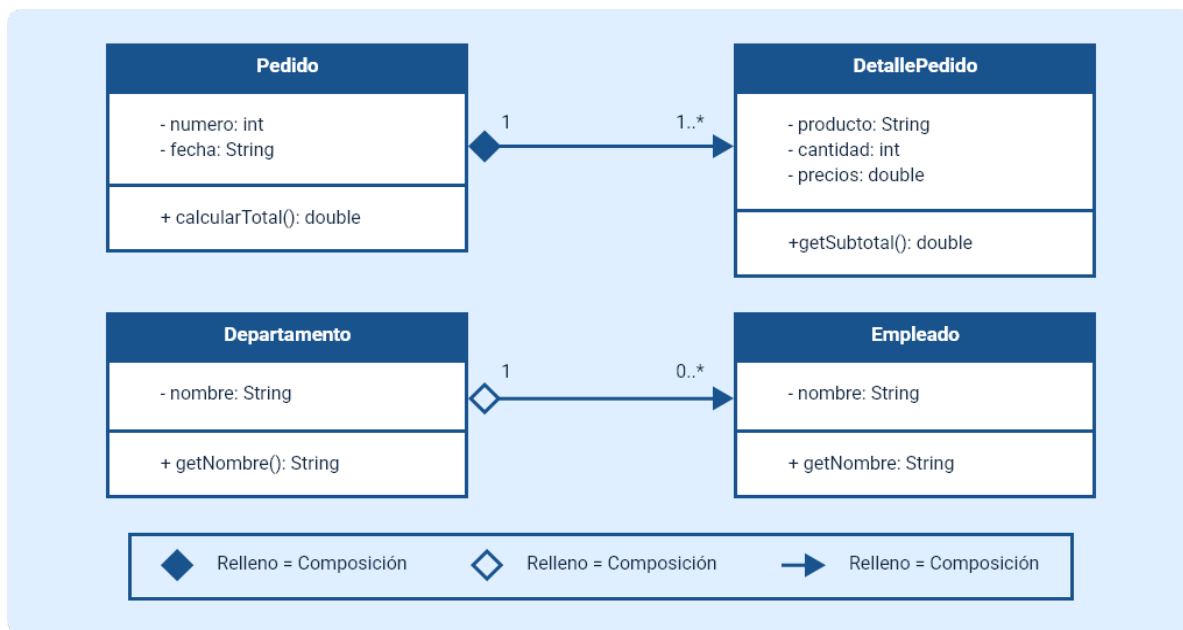
En el diagrama se puede notar que los atributos: codigo, nombre y promedio están marcados con el símbolo - (privado), lo que indica que solo pueden ser accedidos desde dentro de la clase. Los métodos públicos (marcados con +) constituyen la interfaz de la clase accesible desde el exterior.

2.3. Relaciones entre clases: asociación, agregación, composición y herencia

Las relaciones entre clases definen cómo se conectan y comunican los elementos del sistema. Cada tipo de relación expresa un nivel diferente de dependencia y acoplamiento entre las clases. La correcta elección del tipo de relación es fundamental para un diseño orientado a objetos robusto y mantenible.

La siguiente figura ilustra los cuatro tipos de relaciones más importantes en los diagramas de clases UML, mostrando la notación gráfica de cada una:

Figura 2. Tipos de relaciones en diagramas de clases UML: composición y agregación.



En la figura se pueden observar dos tipos de relaciones: la composición (rombo relleno negro) entre Pedido y DetallePedido, donde el DetallePedido no puede existir sin su Pedido; y la agregación (rombo vacío) entre Departamento y Empleado, donde los empleados pueden existir independientemente del departamento.

- **Asociación**

La asociación es la relación más general entre dos clases. Indica que los objetos de una clase conocen o utilizan objetos de otra, sin que ninguna dependa existencialmente de la otra. En Java se implementa mediante un atributo que contiene una referencia a la otra clase. La multiplicidad (`1..1`, `0..*`, `1..*`) indica cuántos objetos participan en cada extremo de la relación. A continuación, su ejemplificación:

```
public class Profesor {
```

```
private String nombre;

private Curso curso; // Asociación con Curso

public void asignarCurso(Curso c) { this.curso = c; }

public Curso getCurso() { return curso; }

}
```

- **Agregación**

La agregación es una relación "todo-parte" débil. El objeto parte puede existir de forma independiente al objeto todo. Los objetos parte son creados externamente y pasados al todo mediante el constructor o setters. Si el todo se destruye, las partes no se destruyen necesariamente. Se puede representar de la siguiente manera:

```
public class Departamento {

    private String nombre;

    private List<Empleado> empleados = new ArrayList<>();

    // El Empleado existe fuera del Departamento (agregación débil)

    public void agregarEmpleado(Empleado e) { empleados.add(e); }

    public void retirarEmpleado(Empleado e) { empleados.remove(e); }

}
```

- **Composición**

La composición es una relación "todo-parte" fuerte. El objeto parte no puede existir sin el todo; su ciclo de vida está ligado al del objeto contenedor. En Java se

implementa creando los objetos parte directamente dentro del constructor o los métodos del todo. Representada de esta manera:

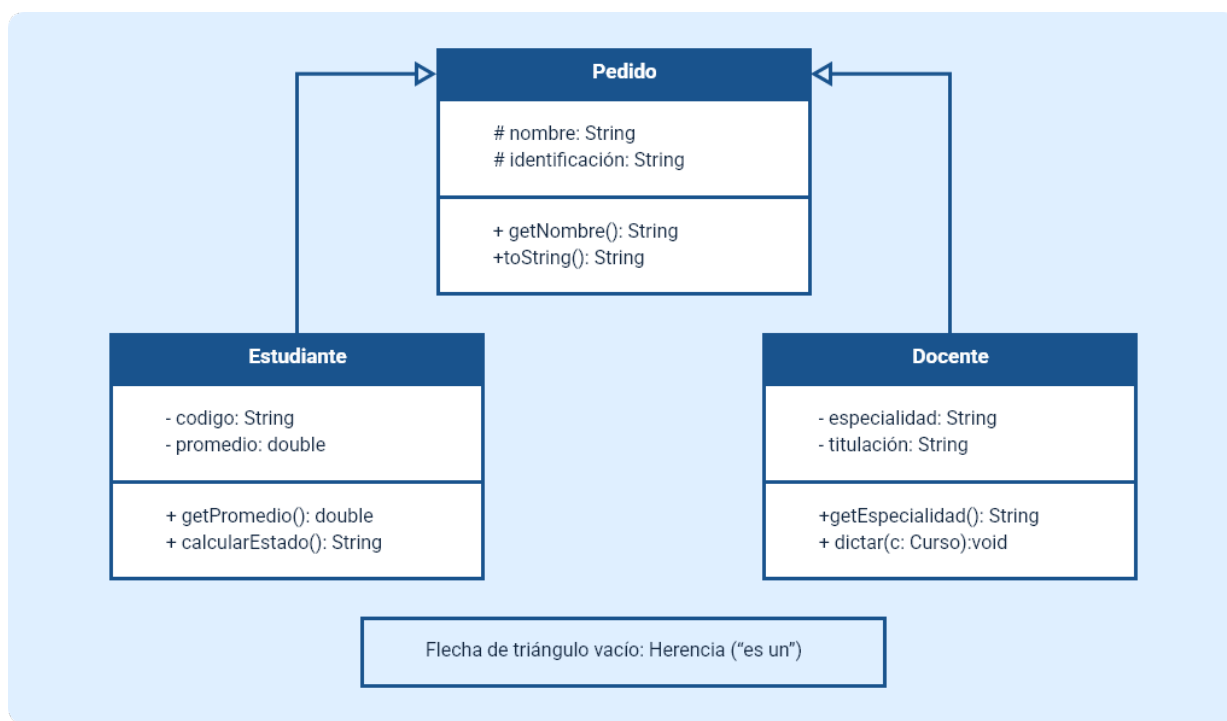
```
public class Pedido {  
  
    private String numero;  
  
    private List<DetallePedido> detalles = new ArrayList<>();  
  
    public Pedido(String numero) { this.numero = numero; }  
  
    // El DetallePedido se crea dentro del Pedido (composición)  
  
    public void agregarProducto(String nombre, int cant, double precio) {  
  
        detalles.add(new DetallePedido(nombre, cant, precio));  
  
    }  
  
    public double calcularTotal() {  
  
        return detalles.stream().mapToDouble(d -> d.getSubtotal()).sum();  
  
    }  
  
}
```

- **Herencia (generalización)**

La relación de herencia o generalización representa la relación "es un" entre clases. La subclase hereda todos los atributos y métodos no privados de la superclase y puede añadir o redefinir comportamientos. En UML se representa con una flecha de triángulo vacío apuntando desde la subclase hacia la superclase.

La siguiente figura muestra la jerarquía de herencia del sistema académico, donde Persona es la clase abstracta base y Estudiante y Docente son sus subclases concretas:

Figura 3. Jerarquía de herencia: Persona (abstracta) con subclases Estudiante y Docente.



La figura muestra que las clases Estudiante y Docente heredan de la clase abstracta Persona (indicada con el nombre en cursiva en UML). Ambas subclases tienen sus propios atributos y métodos adicionales, y sobrescriben el método toString() de la clase base.

La siguiente tabla resume los cuatro tipos de relaciones UML con su notación, la dependencia de vida entre los objetos involucrados y la forma de implementación en Java:

Tabla 3. Tipos de relaciones UML y su implementación en Java.

Tipo de relación	Símbolo UML	Dependencia de vida	Implementación Java
Asociación.	Línea sólida con flecha.	No (independientes).	Atributo referencia.
Agregación.	Rombo vacío (blanco).	No (partes independientes).	Atributo / setter externo.
Composición.	Rombo relleno (negro).	Sí (partes dependen del todo).	new interno al todo.
Herencia.	Flecha triangular vacía.	Sí (jerarquía de tipos).	extends / implements.

2.4. Herramientas CASE para el diseño de diagramas de clases

Las herramientas CASE (Computer-Aided Software Engineering) son aplicaciones de software que automatizan y asisten las tareas de análisis, diseño y documentación en el proceso de desarrollo de sistemas. En el contexto de Java y UML, estas herramientas permiten crear diagramas de clases de forma visual, generar código esqueleto Java a partir del diagrama (ingeniería directa) y crear diagramas UML a partir de código Java existente (ingeniería inversa).

Las herramientas CASE más utilizadas en proyectos Java son:

- **StarUML:** herramienta multiplataforma de código abierto que soporta UML 2.x. Genera código esqueleto Java e importa/exporta en formato XML. Es la opción más recomendada para entornos académicos por su accesibilidad y funcionalidades.
- **Visual Paradigm:** plataforma profesional con integración directa a IntelliJ IDEA y Eclipse. Ofrece ingeniería inversa del código Java a UML y soporte para metodologías ágiles.
- **IntelliJ IDEA (integrado):** el IDE más popular para Java incluye visualización de diagramas de clases directamente desde el código fuente, sin necesidad de herramientas externas.
- **Lucidchart:** herramienta colaborativa en línea que permite crear diagramas UML en tiempo real con múltiples usuarios simultáneos. Ideal para equipos de desarrollo ágiles y distribuidos.
- **Draw.io (diagrams.net):** aplicación web gratuita con soporte UML, integrada en Google Drive, Confluence y GitHub. Es muy utilizada en equipos que trabajan en la nube.

La siguiente tabla presenta una comparación de estas herramientas según criterios relevantes para entornos académicos y profesionales:

Tabla 4. Comparativo de herramientas CASE para diseño UML con Java.

Herramienta	Costo	Gen. código Java	Ingeniería inversa	Colaboración online
StarUML	Gratuito / bajo	Sí	Parcial	No
Visual Paradigm	Pago / trial	Sí	Sí	Limitada

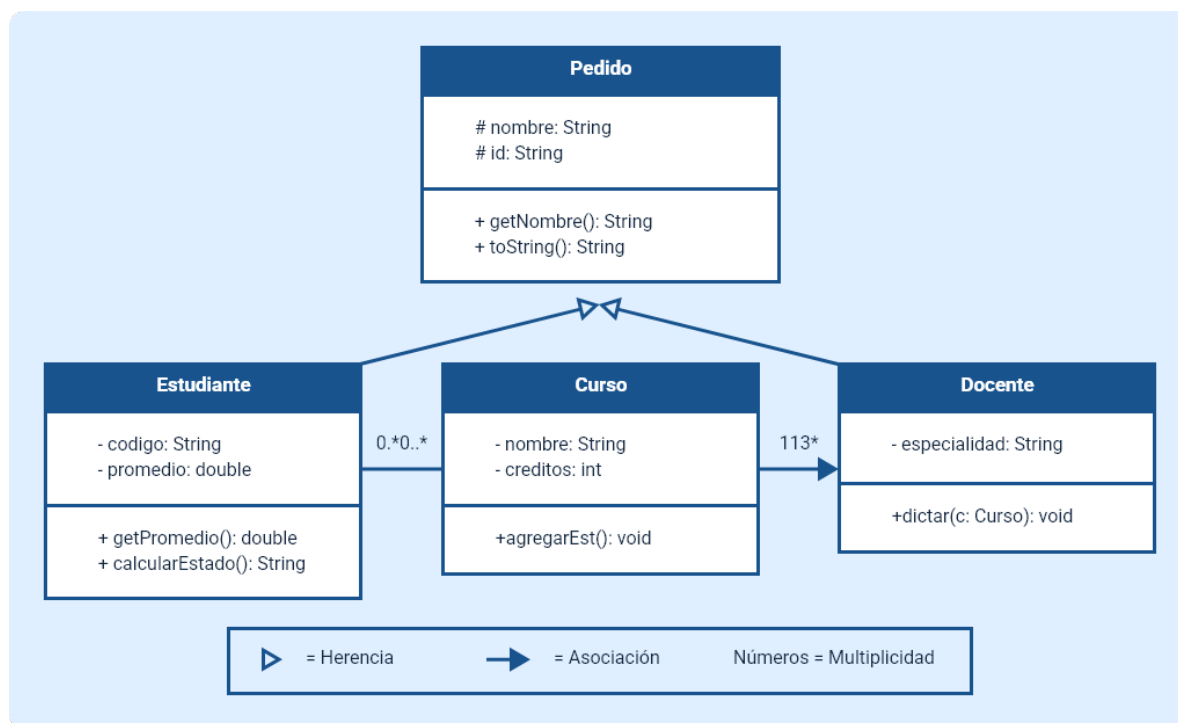
Herramienta	Costo	Gen. código Java	Ingeniería inversa	Colaboración online
IntelliJ IDEA	Gratis / Pro	Indirecta	Sí	No
Lucidchart	Freemium	No	No	Sí
Draw.io	Gratuito	No	No	Sí

2.5. Ejemplo integrador: sistema de gestión académica

Para consolidar los conceptos de modelado UML, se presenta el diagrama de clases completo del sistema de gestión académica que se implementará en Java a lo largo de este componente formativo. Este diagrama ilustra la aplicación simultánea de los cuatro tipos de relaciones y la notación UML completa.

Es por ello, que la siguiente figura presenta el diagrama de clases completo del sistema, incluyendo las relaciones de herencia, asociación y sus multiplicidades:

Figura 4. Diagrama de clases UML completo del sistema de gestión académica.



Persona

`# nombre: String`

`# id: String`

`+ getNombre(): String`

`+ toString(): String`

Estudiante

`- codigo: String`

`- promedio: double`

`+ getPromedio(): double`

+ calcularEstado(): String

Curso

- nombre: String

- creditos: int

+agregarEst(): void

Docente

- especialidad: String

+dictar(c: Curso): void

El diagrama muestra la clase abstracta Persona como base de la jerarquía. Las clases Estudiante y Docente la extienden mediante herencia. Un Docente puede dictar entre 1 y 3 Cursos (asociación 1..3), y un Curso puede tener de 0 a muchos Estudiantes matriculados (asociación 0..*). Este modelo será la base del proyecto integrador desarrollado en la sección 6 del presente componente.

3. Encapsulamiento y control de acceso en clases

El encapsulamiento y el control de acceso son mecanismos fundamentales del paradigma orientado a objetos que garantizan la integridad, seguridad y correcta organización de los datos dentro de un sistema Java. Su adecuada implementación es uno de los factores que más influye en la calidad y mantenibilidad del software, como señala Bloch (2018) en su obra de referencia sobre las mejores prácticas de Java.

3.1. Encapsulamiento en programación orientada a objetos

El encapsulamiento consiste en ocultar los detalles internos de una clase y exponer únicamente lo necesario para interactuar con ella desde el exterior. Este mecanismo, también conocido como "ocultamiento de información" (information hiding), protege los datos de modificaciones no autorizadas y define una interfaz clara y controlada para el uso de los objetos.

En Java, el encapsulamiento se implementa declarando los atributos de la clase como `private` y proporcionando métodos públicos (getters y setters) que controlen el acceso. De esta forma, ningún código externo puede modificar directamente los atributos; debe hacerlo a través de los métodos controlados, donde se puede incorporar lógica de validación.

El siguiente ejemplo muestra una clase `CuentaBancaria` completamente encapsulada, donde el saldo solo puede modificarse a través de métodos que validan las operaciones:

```
public class CuentaBancaria {
```

```
private String numeroCuenta;

private double saldo;

private String titular;

public CuentaBancaria(String numero, String titular, double saldoInicial) {

    this.numeroCuenta = numero;

    this.titular = titular;

    this.saldo = (saldoInicial >= 0) ? saldoInicial : 0;

}

public double getSaldo()    { return saldo; }

public String getNumeroCuenta() { return numeroCuenta; }

public String getTitular()   { return titular; }

public void depositar(double monto) {

    if (monto <= 0) throw new IllegalArgumentException("El monto debe ser
positivo.");

    saldo += monto;

}

public void retirar(double monto) {

    if (monto <= 0) throw new IllegalArgumentException("El monto debe ser
positivo.");
```

```
        if (monto > saldo) throw new IllegalStateException("Saldo insuficiente.");  
  
        saldo -= monto;  
  
    }  
  
}
```

Los beneficios del encapsulamiento en el desarrollo de aplicaciones Java son:

- **Control del estado interno**

El encapsulamiento permite ejercer un control total sobre la modificación del estado interno de un objeto, restringiendo el acceso directo a sus atributos. De esta manera, se evitan estados inválidos o inconsistentes y se garantiza que los cambios se realicen únicamente a través de métodos definidos y controlados.

- **Reducción del acoplamiento**

Al limitar el acceso directo a los datos internos, el encapsulamiento reduce el acoplamiento entre clases. Esto facilita el mantenimiento del sistema, ya que los cambios en una clase tienen un impacto mínimo sobre las demás, mejorando la estabilidad y escalabilidad del proyecto.

- **Validación y protección de datos**

El encapsulamiento permite validar los datos antes de ser almacenados o modificados. Mediante métodos de acceso controlados, se asegura que la información cumpla con las reglas del negocio, preservando la integridad y coherencia de los datos.

- **Flexibilidad en la implementación**

Gracias al encapsulamiento, es posible cambiar la implementación interna de una clase sin afectar al código que la utiliza. Esto permite optimizar o refactorizar el diseño interno sin alterar la interfaz pública ni generar errores en otros componentes.

- **Mejor organización del código**

El encapsulamiento contribuye a una mejor legibilidad y organización del código fuente. Al separar claramente los datos de su forma de acceso, el proyecto resulta más comprensible, ordenado y fácil de analizar para otros desarrolladores.

3.2. Modificadores de acceso público, privado y protegido

Java ofrece cuatro niveles de control de acceso para los miembros de una clase (atributos, métodos y clases anidadas): `public`, `protected`, sin modificador (acceso de paquete) y `private`. Cada nivel define un ámbito diferente de visibilidad y acceso; además, cada uno se puede explicar de la siguiente manera:

- **Public:** los miembros declarados como `public` se conocen como públicos y son accesibles desde cualquier clase, en cualquier paquete del proyecto. Se utilizan principalmente para definir la interfaz pública de una clase; es decir, aquellos métodos o atributos que el código externo puede invocar de manera legítima. Su uso debe ser cuidadoso, ya que una exposición excesiva puede comprometer el control interno y la integridad de los datos.
- **Private:** los miembros declarados como `private` son los de tipo privado y solo son accesibles desde dentro de la propia clase. No pueden ser accedidos

directamente desde subclases ni desde ningún código externo. Este nivel de acceso es el más restrictivo y se recomienda especialmente para los atributos de datos, ya que refuerza el principio de encapsulamiento, obligando a que la manipulación de la información se realice mediante métodos controlados.

- **Protected:** los miembros declarados como `protected`, que hacen alusión a los de nivel protegido, son accesibles desde la propia clase, desde sus subclases (incluso si se encuentran en otros paquetes) y desde las clases del mismo paquete. Este nivel es especialmente útil en jerarquías de herencia, donde las subclases necesitan reutilizar o extender el comportamiento definido en la clase padre sin exponerlo completamente al resto del sistema.
- **Sin modificador (acceso de paquete):** cuando no se especifica ningún modificador de acceso, el miembro es accesible únicamente desde las clases que pertenecen al mismo paquete. No puede ser utilizado por clases externas al paquete, aunque estas sean subclases. Este nivel de acceso permite una cohesión interna entre clases relacionadas, facilitando la colaboración dentro de un módulo o componente del sistema, sin exponer los detalles a otras partes del proyecto. Es el nivel de acceso por defecto en Java.

Basado en lo anterior, a continuación, se presenta un ejemplo práctico en Java que ilustra el uso de los diferentes modificadores. En él se interpreta cómo estos niveles influyen en la visibilidad de atributos y métodos entre una clase base y una subclase, así como las restricciones de acceso según el contexto y el paquete al que pertenecen las clases:

```
public class Persona {  
  
    private int edad;           // Solo accesible en Persona
```

```
protected String nombre;    // Accesible en Persona y subclases

String pais;                // Acceso de paquete (sin modificador)

public String email;        // Accesible desde cualquier parte


public void setEdad(int e) { // Método público de control

    if (e >= 0 && e <= 120) this.edad = e;

}

public int getEdad() { return edad; }

}

public class Estudiante extends Persona {

    public void mostrar() {

        System.out.println(nombre); // OK: protected

        System.out.println(pais);   // OK: mismo paquete

        // System.out.println(edad); // ERROR: private, no accesible

    }

}
```

De igual manera, la siguiente tabla resume el alcance de visibilidad de cada modificador de acceso en Java según el contexto desde donde se accede:

Tabla 5. Alcance de visibilidad de los modificadores de acceso en Java.

Modificador	Misma clase	Subclase / mismo paquete	Código externo
public	Sí	Sí	Sí
protected	Sí	Sí	No (otro paquete)
(paquete)	Sí	Solo mismo paquete	No
private	Sí	No	No

3.3. Getters, setters y el patrón de encapsulamiento completo

El patrón de encapsulamiento completo, conocido en el ecosistema Java como el estándar JavaBeans, consiste en declarar todos los atributos de una clase como `private` e implementar métodos de acceso públicos (getters y setters) con lógica de validación. Este es el estándar de la industria Java adoptado por frameworks como Spring y JPA/Hibernate.

Los getters (métodos de lectura) retornan el valor de un atributo privado. Por convención, su nombre comienza con `get` seguido del nombre del atributo con la primera letra en mayúscula (por ejemplo, `getNombre()`). Los setters (métodos de escritura) asignan un nuevo valor al atributo privado, incorporando las validaciones necesarias para garantizar que el estado del objeto siempre sea válido.

A continuación, se da un claro ejemplo de estos métodos:

```
public class Empleado {

    private String nombre;
```



```
private double salario;

private String cargo;

public Empleado(String nombre, double salario, String cargo) {

    setNombre(nombre); // Se usan los setters para validar

    setSalario(salario);

    setCargo(cargo);

}

// Getters

public String getNombre() { return nombre; }

public double getSalario() { return salario; }

public String getCargo() { return cargo; }


// Setters con validación

public void setNombre(String n) {

    if (n == null || n.trim().isEmpty())

        throw new IllegalArgumentException("El nombre no puede estar vacío.");

    this.nombre = n;

}

public void setSalario(double s) {
```

```
        if (s < 1_000_000 || s > 50_000_000)

            throw new IllegalArgumentException("Salario fuera del rango permitido.");

        this.salario = s;
    }

    public void setCargo(String c) {

        if (c == null || c.trim().isEmpty())

            throw new IllegalArgumentException("El cargo no puede estar vacío.");

        this.cargo = c;
    }

    @Override

    public String toString() {

        return "Empleado{nombre=" + nombre + ", salario=" + salario + ", cargo=" +
cargo + "}";

    }

}
```

3.4. Clases anidadas e internas en Java

Java permite definir clases dentro de otras clases. Estas se denominan nested classes conocidas como clases anidadas o inner classes que son entendidas como las clases internas. Son útiles para encapsular lógica auxiliar estrechamente relacionada

con la clase externa, implementar patrones de diseño como Builder e Iterator y organizar mejor el código cuando una clase pequeña es usada exclusivamente por otra.

Java soporta cuatro tipos de clases anidadas: las clases anidadas estáticas (escritas como static nested class), las clases internas no estáticas (escritas como inner class), las clases locales (definidas dentro de un método) y las clases anónimas (sin nombre, usadas para implementaciones puntuales). El siguiente ejemplo ilustra una clase anidada estática en el contexto del patrón Builder:

```
public class Factura {  
  
    private String numero;  
  
    private String cliente;  
  
    private List<ItemFactura> items;  
  
    // Clase anidada estática: no requiere instancia de Factura  
  
    public static class ItemFactura {  
  
        private String descripcion;  
  
        private int cantidad;  
  
        private double precioUnitario;  
  
        public ItemFactura(String desc, int cant, double precio) {  
  
            this.descripcion = desc;  
  
            this.cantidad = cant;  
  
            this.precioUnitario = precio;  

```

```
}

    public double getSubtotal() { return cantidad * precioUnitario; }

}

public Factura(String numero, String cliente) {

    this.numero = numero;

    this.cliente = cliente;

    this.items = new ArrayList<>();

}

public void agregarItem(String desc, int cant, double precio) {

    items.add(new ItemFactura(desc, cant, precio));

}

public double calcularTotal() {

    return items.stream().mapToDouble(ItemFactura::getSubtotal).sum();

}

}
```

4. Implementación de la programación orientada a objetos en Java

La implementación del paradigma orientado a objetos en Java comprende la traducción de los diseños UML a código funcional. Este proceso requiere dominar los mecanismos propios del lenguaje: codificación de clases desde el diagrama, herencia con `extends` y `super`, interfaces, polimorfismo dinámico con `@Override` y sobrecarga de métodos. Horstmann (2019) describe este proceso como el "puente entre el diseño y el código", resaltando la importancia de la coherencia entre el modelo UML y su implementación.

4.1. Codificación de diagramas de clases

Codificar un diagrama de clases implica convertir cada elemento del diagrama UML en código Java equivalente. El proceso sigue una secuencia lógica que garantiza que la implementación refleje fielmente el diseño: identificar cada clase del diagrama y crear su archivo Java correspondiente; declarar los atributos con el modificador de acceso indicado en el diagrama y el tipo de dato correcto; implementar el constructor o constructores necesarios; codificar los métodos definidos en el diagrama; y finalmente, establecer las relaciones entre clases mediante referencias, colecciones o herencia según corresponda.

El siguiente ejemplo muestra la codificación completa de un diagrama UML que contiene las clases `Producto` e `Inventario`, relacionadas mediante composición:

```
import java.util.ArrayList;  
  
import java.util.List;
```

```
public class Producto {  
  
    private String codigo;  
  
    private String nombre;  
  
    private double precio;  
  
    private int stock;  
  
    public Producto(String codigo, String nombre, double precio, int stock) {  
  
        this.codigo = codigo;  
  
        this.nombre = nombre;  
  
        this.precio = precio;  
  
        this.stock = stock;  
  
    }  
  
    public String getCodigo() { return codigo; }  
  
    public String getNombre() { return nombre; }  
  
    public double getPrecio() { return precio; }  
  
    public int  getStock() { return stock; }  
  
    public void ajustarStock(int cantidad) {  
  
        if (stock + cantidad < 0) throw new IllegalArgumentException("Stock  
insuficiente.");  
  
        stock += cantidad;  
    }  
}
```

```
}

@Override

public String toString() {

    return "[" + codigo + "]" + nombre + " - $" + precio + " (Stock: " + stock + ")";

}

}

public class Inventario {

    private String nombre;

    private List<Producto> productos = new ArrayList<>();

    public Inventario(String nombre) { this.nombre = nombre; }

    public void agregarProducto(Producto p) { productos.add(p); }

    public Producto buscarPorCodigo(String codigo) {

        return productos.stream()

            .filter(p -> p.getCodigo().equals(codigo))

            .findFirst().orElse(null);

    }

    public void mostrarInventario() {

        System.out.println("=== Inventario: " + nombre + " ===");

        productos.forEach(System.out::println);

    }

}
```

```
}  
  
}
```

4.2. Implementación de la herencia en objetos

En Java, la herencia se implementa con la palabra clave `extends` en la declaración de la subclase. La subclase hereda todos los atributos y métodos con visibilidad `public`, `protected` y de paquete de la superclase, pero no los `private`. El constructor de la superclase se invoca explícitamente con `super()` como primera instrucción del constructor de la subclase. Si no se invoca explícitamente, Java llama automáticamente al constructor sin parámetros de la superclase.

El método `super()` también puede usarse para invocar métodos de la superclase desde la subclase, lo que permite reutilizar el comportamiento del padre y extenderlo en lugar de reemplazarlo completamente.

Basado en lo anterior, se presenta un ejemplo práctico en Java que muestra la implementación de la herencia mediante `extends`, el uso de `super()` y la reutilización de métodos de la clase padre:

```
public class Persona {  
  
    protected String nombre;  
  
    protected String identificacion;  
  
    public Persona(String nombre, String identificacion) {  
  
        this.nombre = nombre;
```



```
        this.identificacion = identificacion;

    }

    public void mostrarInfo() {

        System.out.println("Nombre: " + nombre + " | ID: " + identificacion);

    }

    @Override

    public String toString() { return nombre + " (" + identificacion + ")"; }

}

public class Estudiante extends Persona {

    private String codigo;

    private double promedio;

    public Estudiante(String nombre, String id, String codigo) {

        super(nombre, id);    // Invoca constructor de Persona

        this.codigo = codigo;

        this.promedio = 0.0;

    }

    @Override

    public void mostrarInfo() {

        super.mostrarInfo();    // Reutiliza el método del padre
```

```
        System.out.println("Código: " + codigo + " | Promedio: " + promedio);  
    }  
  
    public String calcularEstado() {  
        return promedio >= 3.0 ? "Aprobado" : "En riesgo académico";  
    }  
  
    public double getPromedio() { return promedio; }  
  
    public void setPromedio(double p) {  
        if (p < 0 || p > 5) throw new IllegalArgumentException("Promedio inválido.");  
        this.promedio = p;  
    }  
}
```

4.3. Reutilización de código, interfaces y tipos de herencia

La reutilización de código es uno de los objetivos principales de la POO y una de sus mayores ventajas frente a la programación estructurada. En Java, la reutilización se logra principalmente mediante la herencia (extends), la composición (atributos que referencian otros objetos) y las interfaces (implements).

- **Tipos de herencia en Java**

Aunque Java solo permite herencia simple entre clases, se pueden construir diferentes tipos de jerarquías que responden a distintas necesidades de diseño:

- **Herencia simple:** una clase derivada extiende exactamente una clase base.
Es el tipo más común y recomendado.
- **Herencia multinivel:** cadena de herencia donde A extiende B, B extiende C.
Ejemplo: Vehiculo - Automovil - AutomovilDeportivo.
- **Herencia jerárquica:** varias subclases heredan de una misma superclase.
Ejemplo: Animal como base para Perro, Gato y Pajaro.

- **Interfaces en Java**

Las interfaces permiten definir contratos de comportamiento que múltiples clases no relacionadas pueden implementar. A diferencia de la herencia de clases, una clase puede implementar tantas interfaces como necesite, lo que proporciona una forma de reutilización sin crear jerarquías rígidas. A continuación, se presenta un ejemplo práctico en Java que ilustra el uso de interfaces como contratos de comportamiento y la implementación múltiple en una misma clase:

```
// Interfaces: contratos de comportamiento

public interface Calculable { double calcularCosto(); }

public interface Reportable { String generarReporte(); }

// Una clase puede implementar múltiples interfaces

public class Contrato extends Documento implements Calculable, Reportable {

    private double valorBase;

    @Override

    public double calcularCosto() { return valorBase * 1.19; }
```

```
@Override

public String generarReporte() {

    return "Contrato #" + getId() + " | Costo: " + calcularCosto();

}

}
```

4.4. Polimorfismo y sobreescritura de métodos

El polimorfismo dinámico en Java se implementa mediante la sobreescritura de métodos con la anotación `@Override`. Cuando se usa una referencia de la superclase para apuntar a un objeto de una subclase, Java determina automáticamente en tiempo de ejecución qué versión del método ejecutar según el tipo real del objeto. Este mecanismo es conocido como enlace dinámico (dynamic binding).

El polimorfismo estático, por su parte, se logra mediante la sobrecarga de métodos (method overloading): múltiples métodos en la misma clase con el mismo nombre, pero diferentes parámetros. El compilador determina cuál versión usar en tiempo de compilación según los argumentos pasados.

Para clarificar el procedimiento anterior, se presenta un ejemplo práctico en Java que muestra el polimorfismo dinámico mediante la sobrescritura de métodos y el polimorfismo estático a través de la sobrecarga, evidenciando cómo Java resuelve cada caso:

// Polimorfismo dinámico:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        List<Animal> animales = new ArrayList<>();  
  
        animales.add(new Perro("Rex"));  
  
        animales.add(new Gato("Luna"));  
  
        animales.add(new Pez("Nemo"));  
  
        // Java llama al método correcto según el tipo real del objeto  
  
        for (Animal a : animales) {  
  
            a.hacerSonido();  
  
        }  
  
    }  
  
}  
  
// Salida:  
  
// Rex dice: ¡Guau!  
  
// Luna dice: ¡Miau!  
  
// Nemo dice: ...(silencio)  
  
// Polimorfismo estático (sobrecarga de métodos):  
  
public class Calculadora {
```

```
public int  sumar(int a, int b)    { return a + b; }

public double sumar(double a, double b) { return a + b; }

public int  sumar(int a, int b, int c){ return a + b + c; }

}
```

Por su parte y para finalizar esta temática, la siguiente tabla resume las diferencias entre los dos tipos de polimorfismo en Java:

Tabla 6. Comparativo entre polimorfismo estático y dinámico en Java.

Característica	Polimorfismo estático (sobrecarga)	Polimorfismo dinámico (@Override)
Momento de resolución	Tiempo de compilación	Tiempo de ejecución
Mecanismo Java	Mismo nombre, distintos parámetros	@Override en subclase
Requiere herencia	No	Sí
Ejemplo representativo	sumar(int) y sumar(double)	Animal.hacerSonido() → Perro
Uso principal	Múltiples tipos de entrada válidos	Jerarquías de clases extensibles

5. Validación y ajustes del programa

La validación y los ajustes del programa son etapas fundamentales del ciclo de vida del software que garantizan que la aplicación funcione correctamente, procese datos válidos y brinde una experiencia adecuada al usuario. Un programa sin controles de validación es susceptible a errores, datos inconsistentes y comportamientos inesperados que pueden afectar negativamente la experiencia del usuario y la integridad del sistema.

5.1. Controles de validación en aplicaciones Java

Los controles de validación son mecanismos implementados en el código que verifican si los datos ingresados o procesados por el programa cumplen con los requisitos definidos antes de continuar con la ejecución. En Java, los controles de validación se implementan principalmente en los métodos setter, los constructores y los métodos de negocio mediante bloques condicionales y el lanzamiento de excepciones.

Existen diferentes tipos de validaciones que se aplican según el tipo de dato y el contexto del sistema:

- **Validación de tipo de dato:** verifica que el dato corresponda al tipo esperado por el sistema. Java es fuertemente tipado, por lo que muchas validaciones de tipo son detectadas en tiempo de compilación.
- **Validación de rango:** asegura que el valor numérico o temporal se encuentre dentro de un intervalo permitido. Ejemplo: la edad debe estar entre 0 y 120 años.

- **Validación de formato:** comprueba que el dato cumpla con un patrón estructural específico. En Java se usan expresiones regulares (clase Pattern) para validar formatos como correos electrónicos, números de teléfono o códigos.
- **Validación de presencia:** verifica que los campos obligatorios no sean null ni estén vacíos. En Java se utiliza null y el método String.isBlank() (Java 11+).
- **Validación de consistencia:** comprueba que los datos sean coherentes entre sí. Ejemplo: la fecha de fin no puede ser anterior a la de inicio.

Clarificando lo anterior, se presenta un ejemplo práctico en Java que ilustra la implementación de controles de validación en métodos setter, mostrando cómo verificar tipos, rangos, formatos y valores obligatorios mediante condiciones y excepciones:

```
public class Estudiante {  
  
    private String nombre;  
  
    private int edad;  
  
    private double promedio;  
  
    private String email;  
  
    public void setNombre(String n) {  
  
        if (n == null || n.isBlank())  
  
            throw new IllegalArgumentException("El nombre es obligatorio.");  
  
        if (n.length() < 2 || n.length() > 100)
```



```
        throw new IllegalArgumentException("El nombre debe tener entre 2 y 100
caracteres.");

        this.nombre = n.trim();
    }

    public void setEdad(int e) {

        if (e < 15 || e > 80)

            throw new IllegalArgumentException("La edad debe estar entre 15 y 80
años.");

        this.edad = e;
    }

    public void setPromedio(double p) {

        if (p < 0.0 || p > 5.0)

            throw new IllegalArgumentException("El promedio debe estar entre 0.0 y
5.0.");

        this.promedio = p;
    }

    public void setEmail(String e) {

        if (e == null || !e.matches("^\\w.-]+@[\\w.-]+\\.\\.[a-z]{2,}$"))

            throw new IllegalArgumentException("Formato de email inválido.");

        this.email = e;
    }
}
```

```
}  
  
}
```

5.2. Manejo de excepciones en Java

Java ofrece un robusto mecanismo de manejo de excepciones basado en los bloques try, catch, finally y la instrucción throw. Las excepciones permiten separar el código de manejo de errores del flujo principal del programa, mejorando su legibilidad y robustez. Según Bloch (2018), el correcto uso de excepciones es uno de los aspectos más importantes para escribir código Java de calidad.

En Java, las excepciones se organizan en una jerarquía de clases: todas heredan de Throwable. Las dos ramas principales son Error (para errores graves del sistema que no deben capturarse) y Exception. Dentro de Exception se distinguen las excepciones verificadas (checked exceptions, que heredan de Exception) y las no verificadas (unchecked exceptions, que heredan de RuntimeException).

Lo anterior se ilustra con el siguiente ejemplo:

```
// Manejo básico de excepciones:
```

```
public static void registrarEstudiante(String nombre, int edad, double promedio) {  
  
    Estudiante e = new Estudiante();  
  
    try {  
  
        e.setNombre(nombre);  
  
        e.setEdad(edad);
```

```
e.setPromedio(promedio);

System.out.println("Estudiante registrado correctamente.");

} catch (IllegalArgumentException ex) {

    System.out.println("Error de validación: " + ex.getMessage());

} finally {

    System.out.println("Proceso de registro finalizado.");

}

}

// Excepción personalizada:

public class EstudianteException extends Exception {

    private String campo;

    public EstudianteException(String campo, String mensaje) {

        super(mensaje);

        this.campo = campo;

    }

    public String getCampo() { return campo; }

}
```

Por su parte, la siguiente tabla resume la jerarquía y tipos de excepciones más utilizados en aplicaciones Java orientadas a objetos:

Tabla 7. Tipos de excepciones en Java y su uso recomendado.

Categoría	Tipo / Clase Java	Descripción y uso
Checked	IOException, SQLException	Obligatorio declarar con throws o capturar con catch.
Unchecked	IllegalArgumentException	Para validaciones de argumentos inválidos.
Unchecked	NullPointerException	Cuando se accede a una referencia null.
Unchecked	IndexOutOfBoundsException	Índice de array o colección fuera de rango.
Personalizada	extends Exception / RuntimeException	Para errores específicos del dominio de negocio.

5.3. Refactorización, pruebas y depuración del programa

Los ajustes y la mejora del programa son actividades continuas que buscan optimizar el funcionamiento, la eficiencia y la calidad del código. Martin (2008) define el código limpio como aquel que es fácil de leer, entender y modificar, y establece que la refactorización continua es la práctica clave para alcanzar y mantener ese estándar.

Es así como se abordan las siguientes tres prácticas fundamentales para garantizar la calidad del software en Java: la depuración de errores, la refactorización del código y la realización de pruebas unitarias. Estas actividades permiten detectar fallos, mejorar la estructura interna del programa y verificar su correcto funcionamiento, contribuyendo al desarrollo de aplicaciones más confiables, mantenibles y alineadas con los principios del código limpio:

- **Depuración - Debugging**

La depuración consiste en identificar y corregir errores (bugs) en el código. Los IDEs Java como IntelliJ IDEA y Eclipse ofrecen depuradores integrados que permiten ejecutar el programa paso a paso, establecer puntos de interrupción (breakpoints), inspeccionar el valor de las variables en tiempo de ejecución y evaluar expresiones. Los tipos de errores más comunes son los errores de sintaxis (detectados por el compilador), los errores de lógica (el programa compila, pero produce resultados incorrectos) y los errores de ejecución (el programa lanza una excepción en tiempo de ejecución).

- **Refactorización**

La refactorización es el proceso de reestructurar el código existente sin cambiar su comportamiento externo observable. Su objetivo es mejorar la legibilidad, reducir la complejidad y facilitar el mantenimiento. IntelliJ IDEA ofrece refactorizaciones automatizadas como renombrar clases/métodos/variables, extraer métodos, mover clases a otros paquetes y eliminar código duplicado, todo con un solo atajo de teclado.

- **Pruebas unitarias con JUnit**

Las pruebas unitarias verifican que cada método o componente del programa funciona correctamente de forma aislada. JUnit 5 es el framework de pruebas estándar en el ecosistema Java. Una prueba unitaria bien escrita sigue el patrón AAA: Arrange (preparar los datos), Act (ejecutar el método bajo prueba) y Assert (verificar el resultado esperado).

Partiendo de las tres prácticas anteriores, se presenta un ejemplo práctico en Java que ilustra la aplicación de pruebas unitarias con JUnit, relacionando cómo validar

el comportamiento de los métodos, verificar resultados esperados y detectar errores mediante el uso de aserciones y manejo de excepciones, como parte de las buenas prácticas de calidad y depuración del código:

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class EstudianteTest {

    @Test

    void testSetPromedioValido() {

        // Arrange

        Estudiante e = new Estudiante();

        // Act

        e.setPromedio(4.5);

        // Assert

        assertEquals(4.5, e.getPromedio(), 0.001);

    }

    @Test

    void testSetPromedioFueraDeRango() {

        Estudiante e = new Estudiante();

        assertThrows(IllegalArgumentException.class, () -> e.setPromedio(6.0));

    }

}
```

```
}  
  
@Test  
  
void testNombreVacioLanzaExcepcion() {  
  
    Estudiante e = new Estudiante();  
  
    assertThrows(IllegalArgumentException.class, () -> e.setNombre(""));  
  
}  
  
}
```

5.4. Buenas prácticas y patrones de diseño básicos

Los patrones de diseño, documentados por Gamma, Helm, Johnson & Vlissides (1994) en su obra "Design Patterns", son soluciones reutilizables a problemas recurrentes en el diseño de software orientado a objetos. Representan la experiencia acumulada de la comunidad de desarrolladores y se clasifican en tres categorías: creacionales, estructurales y de comportamiento.

Partiendo de lo anterior, se presentan algunos patrones de diseño básicos ampliamente utilizados en Java, los cuales ofrecen soluciones estructuradas y reutilizables a problemas comunes del desarrollo orientado a objetos. Estos patrones facilitan la creación de software flexible, mantenible y extensible, al aplicar principios de buen diseño y buenas prácticas reconocidas en la ingeniería de software:

- **Patrón Singleton (creacional)**

Garantiza que una clase tenga exactamente una única instancia durante toda la ejecución del programa y proporciona un punto de acceso global a ella. Es ampliamente utilizado para gestores de configuración, conexiones a bases de datos y registros de auditoría.

- **Patrón Factory Method (creacional)**

Delega la creación de objetos a un método estático o una subclase, desacoplando el código cliente de las clases concretas que se instancian. Esto mejora la extensibilidad del sistema y aplica el principio Open/Closed de SOLID.

- **Patrón Observer (comportamiento)**

Define una dependencia uno a muchos entre objetos. Cuando el objeto observado (sujeto) cambia de estado, todos sus observadores son notificados automáticamente. Es el fundamento de los sistemas de eventos, las interfaces gráficas (listeners en Java Swing/JavaFX) y la arquitectura orientada a eventos.

- **Patrón Strategy (comportamiento)**

Define una familia de algoritmos encapsulados en clases separadas e intercambiables. El cliente puede cambiar el algoritmo en tiempo de ejecución sin modificar su código. Aplica directamente los principios Open/Closed e Inversión de Dependencias de SOLID.

- **Patrón Builder (creacional)**

Separa la construcción de un objeto complejo de su representación, permitiendo que el mismo proceso de construcción pueda crear diferentes representaciones. Es muy

utilizado en Java para construir objetos con muchos parámetros opcionales, evitando constructores con demasiados argumentos.

6. Proyecto integrador: sistema de registro académico

El proyecto integrador consolida todos los conceptos estudiados en el componente formativo, mediante el desarrollo de un sistema de registro académico completo en Java. Este sistema aplica simultáneamente los principios de diseño orientado a objetos: clases abstractas, herencia, polimorfismo, encapsulamiento, validación de datos y manejo de excepciones. El modelo de clases del sistema corresponde al diagrama UML presentado en la sección 2.5.

- **Estructura del sistema**

El sistema de registro académico está compuesto por las siguientes clases organizadas en una jerarquía de herencia con relaciones de composición y asociación: la clase abstracta Persona como base, con las subclases concretas Estudiante y Docente; la clase Curso como contenedor de estudiantes; y la clase SistemaAcademico como punto de entrada y coordinación del sistema.

A continuación, se presenta el ejemplo de código que ilustra esta estructura del sistema:

```
// Clase abstracta base

public abstract class Persona {

    private String nombre;

    private String identificacion;

    public Persona(String nombre, String identificacion) {

        setNombre(nombre);
```

```
this.identificacion = identificacion;

}

public void setNombre(String n) {

    if (n == null || n.isBlank())

        throw new IllegalArgumentException("El nombre es obligatorio.");

    this.nombre = n.trim();

}

public String getNombre()    { return nombre; }

public String getIdentificacion() { return identificacion; }

// Método abstracto: cada subclase define su propia representación

public abstract String obtenerRol();

@Override

public String toString() {

    return "[" + obtenerRol() + "]" + " " + nombre + " (" + identificacion + ")";

}

}

public class Estudiante extends Persona {

    private String codigoEstudiante;

    private double promedio;
```

```
public Estudiante(String nombre, String id, String codigo) {  
  
    super(nombre, id);  
  
    this.codigoEstudiante = codigo;  
  
    this.promedio = 0.0;  
  
}  
  
@Override public String obtenerRol() { return "ESTUDIANTE"; }  
  
public double getPromedio() { return promedio; }  
  
public void setPromedio(double p) {  
  
    if (p < 0 || p > 5)  
  
        throw new IllegalArgumentException("Promedio fuera de rango (0-5).");  
  
    this.promedio = p;  
  
}  
  
public String calcularEstado() {  
  
    if (promedio >= 4.0) return "Excelente";  
  
    if (promedio >= 3.0) return "Aprobado";  
  
    return "En riesgo académico";  
  
}  
  
@Override  
  
public String toString() {
```

```
return super.toString() + " | Código: " + codigoEstudiante  
  
    + " | Promedio: " + promedio + " | Estado: " + calcularEstado();  
  
}  
  
}
```

- **Implementación de Docente y Curso**

Como complemento a la jerarquía definida a partir de la clase abstracta Persona y su especialización en Estudiante, se presenta la implementación de la clase Docente y la clase Curso, las cuales amplían y articulan el funcionamiento del sistema de registro académico.

La clase Docente hereda de Persona y define atributos propios como la especialidad y la titulación, reforzando el uso de herencia y polimorfismo mediante la sobreescritura del método obtenerRol().

Por su parte, la clase Curso representa una entidad central del sistema y establece relaciones de composición y asociación, al vincular un docente responsable y una colección de estudiantes, incorporando además validaciones de negocio como el control de cupo y el cálculo del promedio general del curso.

A continuación, se muestra el ejemplo de código que ilustra la implementación de estas clases dentro del sistema académico:

```
public class Docente extends Persona {  
  
    private String especialidad;  
  
    private String titulacion;
```

```
public Docente(String nombre, String id, String especialidad, String titulacion) {  
    super(nombre, id);  
    this.especialidad = especialidad;  
    this.titulacion = titulacion;  
}  
  
@Override public String obtenerRol() { return "DOCENTE"; }  
  
public String getEspecialidad() { return especialidad; }  
  
@Override  
public String toString() {  
    return super.toString() + " | Especialidad: " + especialidad  
        + " | Titulación: " + titulacion;  
}  
}  
  
public class Curso {  
    private String nombre;  
    private int creditos;  
    private Docente docente;  
    private List<Estudiante> estudiantes = new ArrayList<>();  
    private static final int CUPO_MAXIMO = 30;
```

```
public Curso(String nombre, int credits, Docente docente) {

    this.nombre = nombre;

    this.credits = credits;

    this.docente = docente;

}

public void matricularEstudiante(Estudiante e) {

    if (estudiantes.size() >= CUPO_MAXIMO)

        throw new IllegalStateException("El curso ha alcanzado el cupo máximo.");

    if (estudiantes.contains(e))

        throw new IllegalArgumentException("El estudiante ya está matriculado.");

    estudiantes.add(e);

}

public double calcularPromedioCurso() {

    return

estudiantes.stream().mapToDouble(Estudiante::getPromedio).average().orElse(0);

}

public void mostrarListado() {

    System.out.println("\n=== Curso: " + nombre + " (" + credits + " créditos)

===");
```

```
        System.out.println("Docente: " + docente.getNombre());

        System.out.println("Estudiantes matriculados: " + estudiantes.size());

        estudiantes.forEach(System.out::println);

        System.out.printf("Promedio del curso: %.2f%n", calcularPromedioCurso());

    }

}
```

- **Programa principal y demostración del sistema**

El programa principal, representado por la clase SistemaAcademico, actúa como punto de entrada y coordinación del sistema de registro académico. En él se integran todas las clases previamente definidas para demostrar su funcionamiento conjunto en un escenario real de uso.

A través del método main, se crean instancias de Docente, Curso y Estudiante, se asignan promedios, se realiza la matrícula de estudiantes y se aplican validaciones y manejo de excepciones para garantizar la consistencia de los datos.

Además, el ejemplo evidencia el uso del polimorfismo, al manejar objetos de distintas subclases (Docente y Estudiante) mediante referencias del tipo Persona, y muestra cómo el sistema genera salidas informativas que reflejan el estado del curso y sus participantes.

A continuación, se presenta el ejemplo de código que ilustra la ejecución completa y la demostración del sistema académico:

```
public class SistemaAcademico {
```



```
public static void main(String[] args) {  
  
    // Crear docente  
  
    Docente docente = new Docente(  
  
        "Dr. Carlos Mendoza", "DOC001",  
  
        "Ingeniería de Software", "PhD"  
  
    );  
  
    // Crear curso  
  
    Curso poo = new Curso("Programación Orientada a Objetos", 3, docente);  
  
    // Crear y matricular estudiantes  
  
    try {  
  
        Estudiante e1 = new Estudiante("Ana Torres", "1001", "EST001");  
  
        e1.setPromedio(4.5);  
  
        Estudiante e2 = new Estudiante("Luis Gómez", "1002", "EST002");  
  
        e2.setPromedio(2.8);  
  
        Estudiante e3 = new Estudiante("María López", "1003", "EST003");  
  
        e3.setPromedio(3.9);  
  
        poo.matricularEstudiante(e1);  
  
        poo.matricularEstudiante(e2);  
  
        poo.matricularEstudiante(e3);  
  
    }  
}
```

```
// Polimorfismo: lista de Personas

List<Persona> personas = new ArrayList<>();

personas.add(docente);

personas.add(e1);

personas.add(e2);

System.out.println("=== Directorio de personas ===");

personas.forEach(p -> System.out.println(p.toString()));

poo.mostrarListado();

} catch (IllegalArgumentException e) {

    System.out.println("Error de datos: " + e.getMessage());

}

}

}
```

Este proyecto integrador demuestra la aplicación coherente de los conceptos del componente: clase abstracta Persona con encapsulamiento y método abstracto obtenerRol(); subclases Estudiante y Docente con herencia (extends y super), sobreescritura de métodos (@Override) y validaciones; clase Curso con asociación a Docente y composición con la lista de Estudiante; y un programa principal con polimorfismo dinámico y manejo de excepciones. El resultado es una aplicación Java estructurada, reutilizable y alineada con los principios SOLID documentados por Martin (2008).

6.1. Reflexión sobre el diseño orientado a objetos

El desarrollo del proyecto integrador permite evidenciar de forma práctica cómo los conceptos teóricos del paradigma orientado a objetos se traducen en decisiones concretas de diseño e implementación en Java. Cada decisión tomada durante el diseño tiene implicaciones directas sobre la calidad, extensibilidad y mantenibilidad del sistema resultante.

La elección de Persona como clase abstracta en lugar de concreta responde al principio de abstracción: en el dominio académico no existen personas genéricas, sino roles específicos (estudiantes, docentes, administrativos). Al declarar Persona como abstracta y obligar a sus subclases a implementar el método obtenerRol(), el diseño garantiza que no se puedan crear instancias de Persona sin un rol definido, lo que refleja fielmente la realidad del dominio.

La composición entre Curso y la lista de Estudiante (`ArrayList<Estudiante>`) fue elegida sobre la agregación porque los estudiantes están matriculados en el curso y, en el contexto del sistema, su gestión está a cargo del objeto Curso. Sin embargo, si el sistema requiriera que los estudiantes pudieran transferirse entre cursos sin perder su identidad, podría considerarse una relación de agregación débil.

El uso de polimorfismo en la lista `List<Persona>` demuestra uno de los beneficios más poderosos del paradigma: el código cliente puede tratar objetos de distintos tipos (Estudiante, Docente) de manera uniforme a través de la referencia de la superclase, sin necesidad de conocer el tipo concreto en tiempo de compilación. Esto hace que el código sea extensible: si se añade una nueva clase Administrativo que extienda Persona, el código existente no necesita modificarse.

Finalmente, la inclusión de validaciones en los setters y el uso de excepciones de tipo `IllegalArgumentException` sigue las recomendaciones de Bloch (2018): las precondiciones de los métodos deben ser documentadas y verificadas, lanzando excepciones apropiadas cuando no se cumplen. Esto garantiza que los objetos del sistema siempre mantengan un estado válido y coherente, independientemente del contexto desde donde se invoquen sus métodos.

6.2. Extensiones propuestas del sistema

El sistema de registro académico presentado puede extenderse aplicando los conceptos avanzados del componente. A continuación, se proponen extensiones que refuerzan el dominio del paradigma orientado a objetos:

- **Agregar la clase Nota:** modelar las notas como objetos asociados a la relación entre Estudiante y Curso, creando una clase Matricula que contenga la lista de notas y calcule el promedio automáticamente.
- **Implementar una interfaz Exportable:** definir una interfaz con el método `exportarCSV()` que implementen tanto Curso como Estudiante, permitiendo exportar datos en formato de texto plano.
- **Aplicar el patrón Observer:** notificar automáticamente al Docente y a la administración cuando un Estudiante alcanza un promedio de riesgo académico (inferior a 3.0).
- **Persistencia de datos:** integrar la serialización de objetos Java (`Serializable`) o una base de datos SQLite con JDBC para persistir el estado del sistema entre ejecuciones.

- **Interfaz gráfica con JavaFX:** construir una interfaz gráfica de usuario que permita gestionar el sistema académico de forma visual, aplicando el patrón MVC (Modelo-Vista-Controlador).

Cada una de estas extensiones aplica uno o más de los conceptos estudiados en el componente y puede ser asignada como actividad de profundización. Su implementación incremental sobre el proyecto integrador base garantiza la coherencia del diseño y permite al aprendiz evidenciar concretamente el valor del paradigma orientado a objetos en el desarrollo de software de calidad.

Síntesis

El componente formativo aborda de manera integral los fundamentos y la aplicación del paradigma de programación orientada a objetos (POO) usando Java como lenguaje de implementación. Inicia con los conceptos esenciales del paradigma: clases, objetos, atributos, métodos, los cuatro pilares (abstracción, encapsulamiento, herencia y polimorfismo), constructores y ciclo de vida de los objetos, clases abstractas, interfaces y los principios SOLID formulados por Martin (2008).

Se profundiza en el modelado de sistemas mediante UML, con cinco diagramas gráficos que ilustran la notación estándar definida por el OMG (2017), los tipos de relaciones entre clases y un ejemplo integrador completo del sistema académico. Se presentan y comparan las principales herramientas CASE para entornos Java, incluyendo StarUML, Visual Paradigm, IntelliJ IDEA, Lucidchart y Draw.io.

El componente trabaja el encapsulamiento y los modificadores de acceso en Java, el patrón JavaBeans de getters/setters con validación según las recomendaciones de Bloch (2018) y las clases anidadas. Posteriormente, se aborda la implementación práctica: codificación de diagramas, herencia con extends y super, interfaces, polimorfismo dinámico con @Override y sobrecarga de métodos.

Finalmente, se estudia la validación con controles específicos para Java, el manejo de excepciones checked y unchecked, las pruebas unitarias con JUnit 5, la refactorización y depuración en entornos IDE, y los patrones de diseño (Singleton, Factory Method, Observer, Strategy y Builder) documentados por Gamma et al. (1994). Todo lo anterior se sintetiza en el proyecto integrador del sistema de registro

académico, que demuestra la aplicación coherente del paradigma en una solución Java estructurada y mantenible.



Glosario

@Override: anotación Java que indica que un método sobrescribe al de la superclase; activa el polimorfismo dinámico.

Abstracción: proceso de identificar las características esenciales de un objeto, ignorando los detalles irrelevantes.

Clase: plantilla que define los atributos y métodos de los objetos creados a partir de ella.

Clase abstracta: clase que no puede instanciarse directamente; define métodos abstractos que sus subclases deben implementar.

Diagrama de clases: representación gráfica UML que muestra las clases, atributos, métodos y relaciones de un sistema.

Encapsulamiento: principio que oculta los datos internos de una clase y los expone solo mediante una interfaz pública controlada.

Herencia: mecanismo por el cual una subclase adquiere atributos y métodos de una superclase mediante la palabra clave extends.

Herramienta CASE: aplicación que automatiza tareas de análisis, diseño y documentación en el proceso de desarrollo de software.

Instancia: objeto concreto creado con new a partir de una clase; tiene su propio estado en memoria.

Interfaz (interface): contrato de comportamiento en Java que las clases implementan con la palabra clave implements.

JUnit: framework de pruebas unitarias estándar en el ecosistema Java; utiliza anotaciones como `@Test` y `@Override`.

Modificador de acceso: palabra reservada (`public`, `private`, `protected`) que controla la visibilidad de atributos y métodos.

Objeto: entidad con estado (atributos) y comportamiento (métodos) propios, creada como instancia de una clase.

Patrón de diseño: solución reutilizable y documentada a un problema recurrente en el diseño de software orientado a objetos.

Polimorfismo: propiedad que permite que objetos distintos respondan diferente al mismo mensaje según su tipo real.

UML: Lenguaje de Modelado Unificado; estándar gráfico del OMG para visualizar y documentar sistemas de software.

Referencias bibliográficas

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley.

Booch, G., Rumbaugh, J. & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley.

Deitel, P. & Deitel, H. (2018). Java How to Program, Early Objects (11th ed.). Pearson.

Gamma, E., Helm, R., Johnson, R. & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Horstmann, C. S. (2019). Core Java, Vol. 1: Fundamentals (11th ed.). Pearson.

Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

Object Management Group. (2017). Unified Modeling Language Specification, Version 2.5.1. OMG.

Pressman, R. S. & Maxim, B. R. (2020). Ingeniería de software: un enfoque práctico (9ª ed.). McGraw-Hill.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico – Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
José Jaime Luis Tang Pinzón	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Manuel Felipe Echavarria Orozco	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Ernesto Navarro Jaimes	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluadora de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Javier Mauricio Oviedo	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima